# Lab Exercise 15- Creating ConfigMaps in Kubernetes

Viraj bhidola
500121825
B2 DevOps

In this lab exercise, you will learn how to create ConfigMaps in Kubernetes using different methods: from literal values, from a file, from an environment file, and from multiple files. ConfigMaps allow you to decouple configuration artifacts from image content to keep containerized applications portable.

**Step 1: Set Up Kubernetes Cluster**

Ensure you have access to a Kubernetes cluster. You can use a local setup with Minikube, kind, or use a cloud-based Kubernetes service.

```
PS C:\Users\ASUS> kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   11d   v1.35.1
```

**Step 2: Create a ConfigMap from Literal Values**

Create a ConfigMap named example-configmap with literal values:

```
kubectl create configmap example-configmap --from-literal=key1=value1 --from-literal=key2=value2
```

```
PS C:\Users\ASUS> kubectl create configmap example-configmap --from-literal=
key1=value1 --from-literal=key2=value2
configmap/example-configmap created
PS C:\Users\ASUS>
```

Verify the ConfigMap:

```
kubectl get configmap example-configmap -o yaml
```

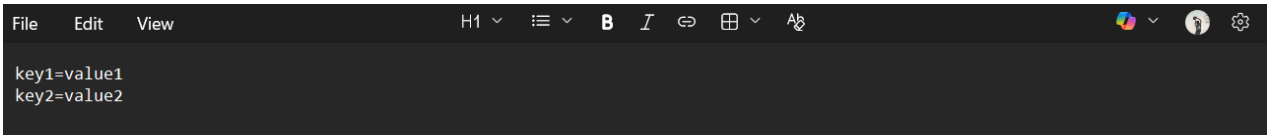
```
PS C:\Users\ASUS> kubectl get configmap example-configmap -o yaml
apiVersion: v1
data:
  key1: value1
  key2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-05T08:24:50Z"
  name: example-configmap
  namespace: default
  resourceVersion: "19553"
  uid: 43c9923e-c5d1-4481-8c0d-30a46d8509a2
PS C:\Users\ASUS>
```

You should see the example-configmap with the keys and values specified.

**Step 3: Create a ConfigMap from a File**

Create a file named **config.txt** with the following content:

```
key1=value1
key2=value2
```




Create a ConfigMap named file-configmap from the file:

```
kubectl create configmap file-configmap --from-file=config.txt
```

```
PS C:\Users\ASUS> kubectl create configmap file-configmap --from-file=config
.txt
configmap/file-configmap created
PS C:\Users\ASUS>
```

Verify the ConfigMap:

```
kubectl get configmap file-configmap -o yaml
```

```
PS C:\Users\ASUS> kubectl get configmap file-configmap -o yaml
apiVersion: v1
data:
  config.txt: "key1=value1\r\nkey2=value2"
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-05T08:27:10Z"
  name: file-configmap
  namespace: default
  resourceVersion: "19664"
  uid: 462d9bd7-c0f9-43b0-9b31-32eeb2b1d530
PS C:\Users\ASUS>
```

You should see the file-configmap with the content of config.txt.

### Step 4: Create a ConfigMap from an Environment File

Create an environment file named **env-config.env** with the following content:

```
ENV_VAR1=value1
ENV_VAR2=value2
```

```
File    Edit    View

ENV_VAR1=value1
ENV_VAR2=value2
```

Create a ConfigMap named env-configmap from the environment file:

```
kubectl create configmap env-configmap --from-env-file=env-config.env
```

```
PS C:\Users\ASUS> notepad env-config.env
PS C:\Users\ASUS> kubectl create configmap env-configmap --from-env-file=env
-config.env
configmap/env-configmap created
```

Verify the ConfigMap:

```
kubectl get configmap env-configmap -o yaml
```

```
PS C:\Users\ASUS> kubectl get configmap env-configmap -o yaml
apiVersion: v1
data:
  ENV_VAR1: value1
  ENV_VAR2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-05T08:30:10Z"
  name: env-configmap
  namespace: default
  resourceVersion: "19812"
  uid: 8da66f2a-f655-4670-8ad1-1292fdd4bde3
PS C:\Users\ASUS>
```

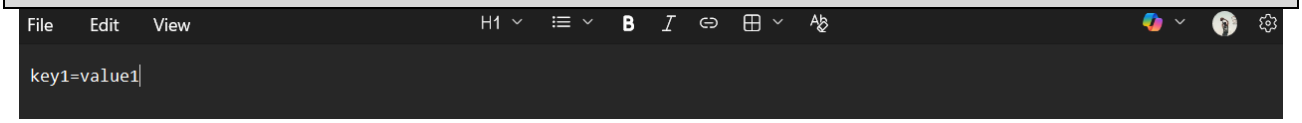
You should see the env-configmap with the environment variables specified in the file.

**Step 5: Create a ConfigMap from Multiple Files**

Create multiple files:

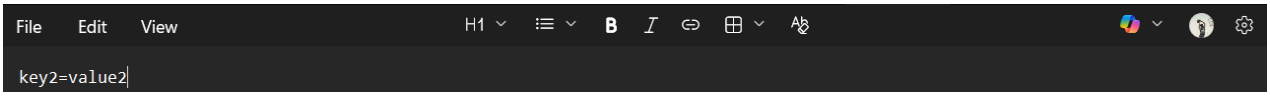
config1.txt with content:

```
key1=value1
```



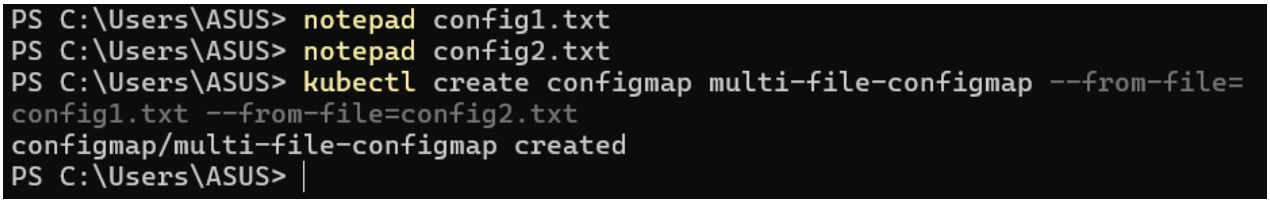

config2.txt with content:

```
key2=value2
```




Create a ConfigMap named multi-file-configmap from multiple files:

```
kubectl create configmap multi-file-configmap --from-file=config1.txt --from-file=config2.txt
```




Verify the ConfigMap:

```
kubectl get configmap multi-file-configmap -o yaml
```

```
PS C:\Users\ASUS> kubectl get configmap multi-file-configmap -o yaml
apiVersion: v1
data:
  config1.txt: key1=value1
  config2.txt: key2=value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-05T08:32:00Z"
  name: multi-file-configmap
  namespace: default
  resourceVersion: "19905"
  uid: 92eaf7ed-1648-42cd-bfcb-a80bd9c99fdd
PS C:\Users\ASUS>
```

You should see the multi-file-configmap with the content of both config1.txt and config2.txt.

**Step 6: Use ConfigMaps in a Pod**

Create a file named **pod-configmap.yaml** with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
  - name: nginx
    image: nginx
    envFrom:
    - configMapRef:
        name: env-configmap
```

```
File    Edit    View

apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
  - name: nginx
    image: nginx
    envFrom:
    - configMapRef:
        name: env-configmap
```

In this manifest:

- The Pod reads environment variables from the env-configmap.
- The Pod mounts the file-configmap at /etc/config.

Apply the manifest to create the Pod:

```
kubectl apply -f pod-configmap.yaml
```

```
PS C:\Users\ASUS> notepad pod-configmap.yaml
PS C:\Users\ASUS> kubectl apply -f pod-configmap.yaml
pod/configmap-pod created
PS C:\Users\ASUS>
```

Check the status of the Pod:

```
kubectl get pods
```

```
PS C:\Users\ASUS> kubectl get pods
NAME            READY   STATUS    RESTARTS   AGE
configmap-pod   1/1     Running   0          17s
PS C:\Users\ASUS>
```

View the Pod logs to see the environment variables and file content:

```
kubectl exec -it configmap-pod -- printenv
```

```
PS C:\Users\ASUS> kubectl exec -it configmap-pod -- printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=configmap-pod
TERM=xterm
ENV_VAR1=value1
ENV_VAR2=value2
NODEPORT_SERVICE_SERVICE_HOST=10.108.246.39
NODEPORT_SERVICE_PORT=tcp://10.108.246.39:80
NODEPORT_SERVICE_PORT_80_TCP=tcp://10.108.246.39:80
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT_443_TCP_PORT=443
NODEPORT_SERVICE_PORT_80_TCP_PORT=80
NODEPORT_SERVICE_PORT_80_TCP_ADDR=10.108.246.39
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
NODEPORT_SERVICE_SERVICE_PORT=80
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
NODEPORT_SERVICE_PORT_80_TCP_PROTO=tcp
NGINX_VERSION=1.29.5
NJS_VERSION=0.9.5
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root
PS C:\Users\ASUS>
```

You should see the environment variables and the content of the file-configmap.

**Step 7: Clean Up**

After completing the exercise, clean up the resources created:

```
kubectl delete pod configmap-pod
kubectl delete configmap example-configmap
kubectl delete configmap file-configmap
```

```
kubectl delete configmap env-configmap
kubectl delete configmap multi-file-configmap
rm config.txt env-config.env config1.txt config2.txt
```

```
PS C:\Users\ASUS> kubectl delete pod configmap-pod
pod "configmap-pod" deleted from default namespace
PS C:\Users\ASUS> kubectl delete configmap example-configmap
configmap "example-configmap" deleted from default namespace
PS C:\Users\ASUS> kubectl delete configmap file-configmap
configmap "file-configmap" deleted from default namespace
PS C:\Users\ASUS> kubectl delete configmap env-configmap
configmap "env-configmap" deleted from default namespace
PS C:\Users\ASUS> kubectl delete configmap multi-file-configmap
configmap "multi-file-configmap" deleted from default namespace
PS C:\Users\ASUS>
```